

Threat Hunt Lab 3: Suspected Data Exfiltration from PIPd Employee

Create a VM and onboard it to MDE if you haven't already — Do not use labuser/Cyberlab123! for credentials (or any other easy password). Your VM will most certainly get breached by a bad actor if it's on long enough. This has already happened once.

Note: "**windows-target-1**" might be called "**windows-target-**" when you query for it. MDE will sometimes cut the name off if it's too long

Create a (Windows) VM if you don't already have one

Log into it and disable the Windows Firewall (start → run → wf.msc → turn firewall off for all profiles), this is just to let the VM be discovered by bad actors on the Internet more easily

Ensure the Network Security Group is WIDE OPEN, allowing all traffic if you want to generate failed login attempt logs.

Onboard it to MDE if it's not already: Onboarding a VM to MDE, Isolation, and Investigation

Ensure it's appearing as onboarded in the MDE Portal: <https://security.microsoft.com/machines>

Ensure the logs are showing up in one of the tables. For example, go to Hunting → Advanced Hunting and search for your device in one of the tables. Example query:

```
DeviceLogonEvents  
| where DeviceName startswith "mydevice-"  
| order by Timestamp desc
```

Example:

```

8 DeviceLogonEvents
9 | where DeviceName startswith "windows-target-"
10 | order by Timestamp desc
11

```

DeviceName	ActionType	LogonType	AccountDomain	AccountName ↓	AccountSid
7473b0cbc4...	LogonFailed	Network		zachary.lang	
ecfb0bad91f...	LogonFailed	Network		webadmin	
ecfb0bad91f...	LogonFailed	Network		webadmin	

Note: "windows-target-1" might be called "windows-target-" when you query for it. MDE will sometimes cut the name off if it's too long
Run this PowerShell command on your VM after onboarding it to MDE

```
Invoke-WebRequest -Uri 'https://raw.githubusercontent.com/joshmadakor1/lognpacific-public/refs/heads/main/cyber-range/entropy-gorilla/exfiltratedata.ps1' -OutFile 'C:\programdata\exfiltratedata.ps1';cmd /c powershell.exe -ExecutionPolicy Bypass -File C:\programdata\exfiltratedata.ps1
```

Investigation Scenario: Data Exfiltration from PIPd Employee

1. Preparation

- **Goal:** Set up the hunt by defining what you're looking for.
 - An employee named John Doe, working in a sensitive department, recently got put on a performance improvement plan (PIP). After John threw a fit, management has raised concerns that John may be planning to steal proprietary information and then quit the company. Your task is to investigate John's activities on his corporate device (windows-target-) using Microsoft Defender for Endpoint (MDE) and ensure nothing suspicious is taking place.
- **Activity:** Develop a hypothesis based on threat intelligence and security gaps (e.g., "Could there be lateral movement in the network?").
 - John is an administrator on his device and is not limited on which applications he uses. He may try to archive/compress sensitive information and send it to a private drive or something.

2. Data Collection

- **Goal:** Gather relevant data from logs, network traffic, and endpoints.
 - Consider inspecting process activity as well as the file system for anything that matches the compression or exfiltration of data.

- **Activity:** Ensure data is available from all key sources for analysis.
 - Ensure the relevant tables contain recent logs for your virtual machine:
 - ◆ DeviceFileEvents
 - ◆ DeviceProcessEvents
 - ◆ DeviceNetworkEvents

3. Data Analysis

- **Goal:** Analyze data to test your hypothesis.
- **Activity:** Look for anomalies, patterns, or indicators of compromise (IOCs) using various tools and techniques.
 - Is there any evidence of anything that resembles company files being archived?
 - If so, can you identify exactly what is happening?
 - If you find something, take note of the time stamp and search the other tables for +/-1 minutes around the same time to see if you can notice anything
 - Take note of your findings with the corresponding queries below

4. Investigation

- **Goal:** Investigate any suspicious findings.
- **Activity:** Dig deeper into detected threats, determine their scope, and escalate if necessary. See if anything you find matches TTPs within the [MITRE ATT&CK Framework](#).
 - Search the DeviceProcessEvents
 - You can use ChatGPT to figure this out by pasting/uploading the logs: [Scenario 4: TTPs](#):

The logs you have provided show several notable actions related to potential data exfiltration. The following MITRE ATT&CK techniques seem to correlate with the activity described:

1. **T1560.001 - Archive Collected Data: Archive via Utility:**
 - The logs mention the use of 7z.exe to archive files such as employee-data.csv into .zip archives. This is a common technique for compressing data before exfiltration, allowing attackers to send out smaller, compressed files.
2. **T1048 - Exfiltration Over Alternative Protocol:**
 - Although the exact network activity isn't shown, the use of archives like .zip files often indicates preparation for exfiltration, potentially using alternative methods for sending the data out of the network, such as through HTTP, HTTPS, or other protocols.
3. **T1059.001 - Command and Scripting Interpreter: PowerShell:**

- PowerShell (powershell.exe) appears frequently in the logs, especially with commands like -ExecutionPolicy Bypass. PowerShell is often used in attacks to execute malicious scripts and orchestrate the exfiltration process. In this case, the script (exfiltratedata.ps1) suggests data extraction and transfer activity.

4. **T1071.001 - Application Layer Protocol: Web Protocols:**

- While the specific protocol isn't listed in these logs, PowerShell scripts like exfiltratedata.ps1 are frequently used to send data over common web protocols such as HTTP or HTTPS, which attackers use to blend in with normal traffic.

5. **T1070.004 - Indicator Removal on Host: File Deletion:**

- Although not directly seen in these logs, attackers often delete or move files after exfiltration to hide their tracks. This may correlate with the process flow of archiving and potential file movements as seen in the logs.

6. **T1027 - Obfuscated Files or Information:**

- The use of archive files (7z.exe) to compress sensitive data, combined with potential encryption in the archive, aligns with efforts to obfuscate exfiltrated data before transferring it out of the environment.

These TTPs together suggest a coordinated exfiltration of sensitive employee data, leveraging PowerShell scripting and archiving utilities to prepare data for removal from the system. Let me know if you'd like a more specific breakdown or further analysis.

5. Response

- **Goal:** Mitigate any confirmed threats.
- **Activity:** Work with security teams to contain, remove, and recover from the threat.
 - Can anything be done?

6. Documentation

- **Goal:** Record your findings and learn from them.
- **Activity:** Document what you found and use it to improve future hunts and defenses.
 - Document what you did

7. Improvement

- **Goal:** Improve your security posture or refine your methods for the next hunt.

- **Activity:** Adjust strategies and tools based on what worked or didn't.
 - Anything we could have done to prevent the thing we hunted for? Any way we could have improved our hunting process?

Notes / Findings:

Sample Queries (spoilers, highlight or copy/paste to reveal):

```
// Look for any kind of archive activity
let archive_applications = dynamic(["winrar.exe", "7z.exe", "winzip32.exe",
"peazip.exe", "Bandizip.exe", "UniExtract.exe", "POWERARC.EXE",
"IZArc.exe", "AshampooZIP.exe", "FreeArc.exe"]);
let VMName = "windows-target-";
DeviceProcessEvents
| where FileName has_any(archive_applications)
| order by Timestamp desc
```

```
// Look for any file activity, based on the Timestamp from any discovered
process activity
let specificTime = datetime(2024-10-15T19:00:48.5615171Z);
let VMName = "windows-target-";
DeviceFileEvents
| where Timestamp between ((specificTime - 1m) .. (specificTime + 1m))
| where DeviceName == VMName
| order by Timestamp desc
```

```
// Look for any network activity, based on the Timestamp from the process or
file activity
let VMName = "windows-target-";
let specificTime = datetime(2024-10-15T19:00:48.5615171Z);
DeviceNetworkEvents
| where Timestamp between ((specificTime - 2m) .. (specificTime + 2m))
| where DeviceName == VMName
| order by Timestamp desc
```

Timeline Summary and Findings:

We did a search within MDE DeviceFileEvents for any activities with zip files, and found a lot of regular activity of archiving stuff and moving to a "backup" folder:

```
DeviceFileEvents
| where DeviceName == "windows-target-1"
| where FileName endswith ".zip"
| order by Timestamp desc
```

Query

Query results are presented in your local time zone as per settings. Kusto filters, however, work in UTC. [Don't want to see it again](#)

```
1 | DeviceFileEvents
2 | where DeviceName == "windows-target-1"
3 | where FileName endswith ".zip"
4 | order by Timestamp desc
```

Getting started **Results** Query history

Export Link to incident Take actions Show empty columns 1 of 682 selected Search 00:00.227 Low Chart type Full screen

Timestamp	DeviceId	DeviceName	ActionType	FileName	FolderPath
18/2024/5/48...	b1856ac7473b0cb04...	windows-target-1	FileRenamed	employee-data-20241017224823.zip	C:\ProgramData\backup\employee-data-20241017224823.zip
18/2024/5/48...	b1856ac7473b0cb04...	windows-target-1	FileCreated	employee-data-20241017224823.zip	C:\ProgramData\employee-data-20241017224823.zip
18/2024/5/36...	b1856ac7473b0cb04...	windows-target-1	FileModified	VMAgentLogs.zip	D:\CollectGuestLogs\temp\VMAgentLogs.zip
18/2024/4/48...	b1856ac7473b0cb04...	windows-target-1	FileRenamed	employee-data-20241017214824.zip	C:\ProgramData\backup\employee-data-20241017214824.zip
18/2024/4/48...	b1856ac7473b0cb04...	windows-target-1	FileCreated	employee-data-20241017214824.zip	C:\ProgramData\employee-data-20241017214824.zip

Getting started **Results** Query history

Export Link to incident Take actions Show empty columns 1 of 682 selected Search 00:00.227 Low Chart type Full screen

Timestamp	DeviceId	DeviceName	ActionType	FileName	FolderPath
18/2024/5/48...	b1856ac7473b0cb04...	windows-target-1	FileRenamed	employee-data-20241017224823.zip	C:\ProgramData\backup\employee-data-20241017224823.zip
18/2024/5/48...	b1856ac7473b0cb04...	windows-target-1	FileCreated	employee-data-20241017224823.zip	C:\ProgramData\employee-data-20241017224823.zip
18/2024/5/36...	b1856ac7473b0cb04...	windows-target-1	FileModified	VMAgentLogs.zip	D:\CollectGuestLogs\temp\VMAgentLogs.zip
18/2024/4/48...	b1856ac7473b0cb04...	windows-target-1	FileRenamed	employee-data-20241017214824.zip	C:\ProgramData\backup\employee-data-20241017214824.zip
18/2024/4/48...	b1856ac7473b0cb04...	windows-target-1	FileCreated	employee-data-20241017214824.zip	C:\ProgramData\employee-data-20241017214824.zip

I took one of the instances of a zip file being created, took the timestamp and searched under DeviceProcessEvents for anything happening 2 minutes before the archive was created and 2 minutes after. I discovered around the same time, a powershell script silently installed 7zip and then used 7zip to zip up employee data into an archive:

```
// 2024-10-18T06:48:43.1958879Z
let VMName = "windows-target-1";
let specificTime = datetime(2024-10-15T19:00:48.5615171Z);
DeviceProcessEvents
| where Timestamp between ({specificTime - 2m} .. {specificTime + 2m})
| where DeviceName == VMName
| order by Timestamp desc
```

Run query

Set in query

Save

Show link

Create detection rule

Query

Query results are presented in your local time zone (per settings). Kusto Maps, Rows, and work in UTC. Don't want to see it again

```
1 DeviceProcessEvents
2 | where DeviceName == "windows-target-1"
3 | where FileName endswith ".zip"
4 | order by Timestamp desc
5
6 // 2024-10-18T06:48:43.1958879Z
7 let VMName = "windows-target-1";
8 let specificTime = datetime(2024-10-15T19:00:48.5615171Z);
9 DeviceProcessEvents
10 | where Timestamp between ({specificTime - 2m} .. {specificTime + 2m})
11 | where DeviceName == VMName
12 | order by Timestamp desc
13 | project Timestamp, DeviceName, ActionType, FileName, ProcessCommandLine
14
```

Getting started Results Query history

Export

Show empty columns

3 items

Search

00:00.146

Low

Chart type

Full screen

Filters:

Add filter

☐ Timestamp	DeviceName	ActionType	FileName	ProcessCommandLine
☐ > Oct 16, 2024 2:00...	windows-target-1	ProcessCreated	7z.exe	"7z.exe" a C:\ProgramData\employee-data-20241015190038.zip C:\ProgramData\employee-data-temp\20241015190038.csv
☐ > Oct 16, 2024 2:00...	windows-target-1	ProcessCreated	7z2408-x64.exe	"7z2408-x64.exe" /S
☐ > Oct 16, 2024 2:00...	windows-target-1	ProcessCreated	powershell.exe	powershell -ExecutionPolicy Unrestricted -File script97.ps1

Getting started Results Query history

Export

Show empty columns

3 items

Search

00:00.146

Low

Chart type

Full screen

Filters:

Add filter

☐ Timestamp	DeviceName	ActionType	FileName	ProcessCommandLine
☐ > Oct 16, 2024 2:00...	windows-target-1	ProcessCreated	7z.exe	"7z.exe" a C:\ProgramData\employee-data-20241015190038.zip C:\ProgramData\employee-data-temp\20241015190038.csv
☐ > Oct 16, 2024 2:00...	windows-target-1	ProcessCreated	7z2408-x64.exe	"7z2408-x64.exe" /S
☐ > Oct 16, 2024 2:00...	windows-target-1	ProcessCreated	powershell.exe	powershell -ExecutionPolicy Unrestricted -File script97.ps1

I searched around the same time period for any evidence of exfiltration from the network, but I didn't see any logs indicating as such:

```
// 2024-10-18T06:48:43.1958879Z
let VMName = "windows-target-1";
let specificTime = datetime(2024-10-18T06:51:50.0992292Z);
DeviceNetworkEvents
| where Timestamp between ({specificTime - 4m} .. {specificTime + 4m})
| where DeviceName == VMName
| order by Timestamp desc
```

Response:

I relayed the information to the employees manager, including everything with the archives [be created at regular intervals via powershell script](#). There didn't appear to be any evidence of exfiltration.